**FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO**

Curso de Ciência da Computação

6º Semestre — Turma 6º CCOMP

# SMASH

## FIRMWARE, COMPONENTES E FUNCIONAMENTO DA PAREDE INSTRUMENTADA DE TREINO DE TÊNIS

Entrega 2 — Sistemas Embarcados e Robótica

**Projeto 7 — Grupo Smash**

Felipe Vallim Soares — 24026060

Guilhermy Mariano Lisboa Garcia — 23025371

Gustavo Oliveira Demetrio — 24026213

Saulo Pereira de Jesus — 24026095

São Paulo

2026

## 1 O PROJETO

O **Smash** transforma uma parede em uma superfície instrumentada de treino de tênis. Seis alvos de MDF, em uma grade de 3 colunas por 2 linhas, têm cada um uma pastilha piezoelétrica colada atrás. Quando um alvo é atingido, o ESP32 identifica qual foi, mede a intensidade do golpe e envia o evento por Wi-Fi (WebSocket) ao aplicativo, que conduz o jogo em tempo real. O LED azul da própria placa é o atuador: pisca para indicar acerto, erro e fim de nível.

## 2 COMPONENTES

Quadro 1 — Componentes da parede.

| Componente | Qtd. | Função |
|---|---|---|
| ESP32 DevKit (38 pinos) | 1 | Lê os sensores, executa a detecção e o servidor WebSocket (Wi-Fi integrado). |
| Piezo de 27 mm | 6 | Sensor de impacto: converte a vibração da placa em tensão. |
| Placa de MDF | 6 | Alvo que recebe a bola. |
| Resistor de 1 MΩ | 6 | Descarrega o piezo para o GND entre uma batida e outra. |
| Diodo 1N4148 | 12 | Dois por piezo: limitam o sinal entre 0 e 3,3 V e protegem o pino do ESP32. |
| LED azul da placa (GPIO 2) | 1 | Atuador: sinaliza acerto, erro e fim de nível ao jogador. |
| Protoboard, jumpers e cabo de par trançado | — | Montagem do circuito e ligação dos piezos ao ESP32. |

Cada piezo tem o seu pino de ADC1: A1 a A6 em IO32, IO33, IO34, IO36, IO39 e IO35, sem multiplexador. Só o ADC1 é usado porque o ADC2 devolve leituras inválidas com o Wi-Fi ligado.

## 3 CÓDIGO UTILIZADO

O firmware foi escrito em C++ com o framework Arduino, para a placa ESP32 Dev Module, e usa duas bibliotecas: WebSockets (Markus Sattler), para o servidor na porta 81, e ArduinoJson (Benoît Blanchon), para montar e ler as mensagens. O código fica no repositório do projeto, https://github.com/2026-2-NCC6/Projeto7, e é dividido em módulos:

Quadro 2 — Módulos do firmware, em src/Entrega1/backend/firmware/smash_wall.

| Arquivo | O que faz |
|---|---|
| `smash_wall.ino` | setup() e loop(): a cada volta, atende a rede, avança o LED e varre os sensores. |
| `config.h` | Pinos, limiar de cada alvo, tempos da detecção e padrões de piscada do LED. |
| `wall_sensors` | Leitura do ADC e máquina de estados da detecção de impacto. |
| `wall_link` | Wi-Fi, mDNS (smash.local), servidor WebSocket e pareamento do app por token. |
| `wall_protocol` | Monta as mensagens JSON (impacto, hello, status) e interpreta os comandos do app. |
| `wall_actuator` | Pisca o LED sem bloquear o loop(), comparando millis(). |

## 4 COMO FUNCIONA

A detecção é uma máquina de estados que não bloqueia o loop():

1. **Repouso.** O ESP32 varre os seis canais sem parar; uma varredura leva menos de 1 ms.
2. **Janela de pico.** Quando um canal passa do limiar do próprio alvo, abre-se uma janela de 20 ms em que o pico dos seis canais é medido.
3. **Decisão.** O alvo atingido é o de maior pico. A vibração que vaza para as placas vizinhas também cruza o limiar, mas com pico menor, e é descartada. Se nem o maior pico chega ao limiar, era ruído do ADC e nada é enviado.
4. **Envio.** O firmware monta o evento `{"alvo":5,"linha":2,"coluna":2,"intensidade":1830,…}` e o envia ao app; o mesmo evento é impresso no Serial.
5. **Debounce.** 200 ms de silêncio impedem que uma batida vire dois eventos.
6. **Retorno.** O app aplica a regra do jogo e responde com `{"cmd":"feedback","resultado":"acerto"}` ou `"erro"`. O LED acende por 250 ms no acerto e pisca três vezes rápido no erro. Sem app conectado, cada batida detectada pisca o LED uma vez.

## 5 O QUE OS VÍDEOS MOSTRAM

**Vídeo 1:** https://drive.google.com/file/d/1hjJLqk6Jfi-lXQAZChO4K6RpVwWxO8Le/view
**Vídeo 2:** https://drive.google.com/file/d/1kquNeE0LlGZBQc6pd13NbqFEeSFYMJYH/view
**Pasta:** https://drive.google.com/drive/folders/114f-HS2OIY9HV7KM-ucOqy8qsduy_-E7

Nos dois vídeos, o app roda no emulador Android do notebook, e o ESP32, na protoboard, está ligado às seis placas de MDF sobre a mesa. Parede e notebook estão na mesma rede Wi-Fi.

**Vídeo 1.** O integrante entra no app como visitante, escolhe um modo na tela inicial e aguarda a contagem regressiva. Em seguida bate com a mão nas placas, e cada batida aparece no app no mesmo instante.

**Vídeo 2.** Partida do modo Infinite Score. A tela mostra a pontuação, a barra de tempo restante e os seis alvos com o valor de cada cor (+10, +20 e +30). A cada batida numa placa, a pontuação sobe na hora.

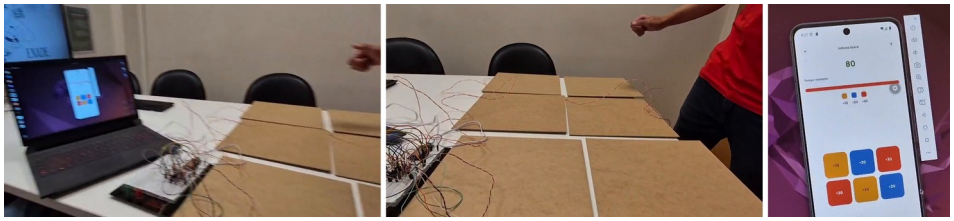


Figura 1 — Quadros dos vídeos: a montagem, uma batida nas placas e o app no modo Infinite Score.